

New MySafe - Account Recovery

senhasegura

Departamento de Segurança e Inovação - SEGi9



CONFIDENCIAL



Resumo Executivo

Este texto aborda o processo de recuperação das chaves de criptografia dos Usuários, no qual o Custodiante designado restaura as chaves de criptografia dos usuários de uma organização. Desta forma, o Custodiante restabelece o acesso em casos de esquecimento ou perda da senha mestra por parte dos Usuários. Essa funcionalidade pode ser ativada para a organização ao habilitar a política de administração de recuperação de chave de criptografia.





Sumário

1	Segurança das Chaves de Criptografia	2
1.1	Alice	2
1.2	Bob	3
2	Custódia das Chaves de Criptografia	6
2.1	Custodiando	6
3	Recuperando Custódia das Chaves	8
3.1	Gerando Nova Chave Mestra - Bob	8
3.2	Nova Custódia - Alice	9





Nota sobre os exemplos: os comandos com AES-CBC abaixo ilustram a proteção de confidencialidade e deixam explícitos chave e IV. Em uma implementação real, cada texto cifrado deve também ser autenticado (por exemplo, com HMAC em chave independente) ou protegido por uma construção AEAD apropriada.





1 Segurança das Chaves de Criptografia

1.1 Alice

Processo de criação das chaves, Pública (KPUAlice.pem), Privada (KPrAlice.pem) e de Dados (DEKAlice.bin), juntamente com o processo de encriptação para armazenamento seguro das chaves Privada (KPrAlice.pem) e de Dados (DEKAlice).

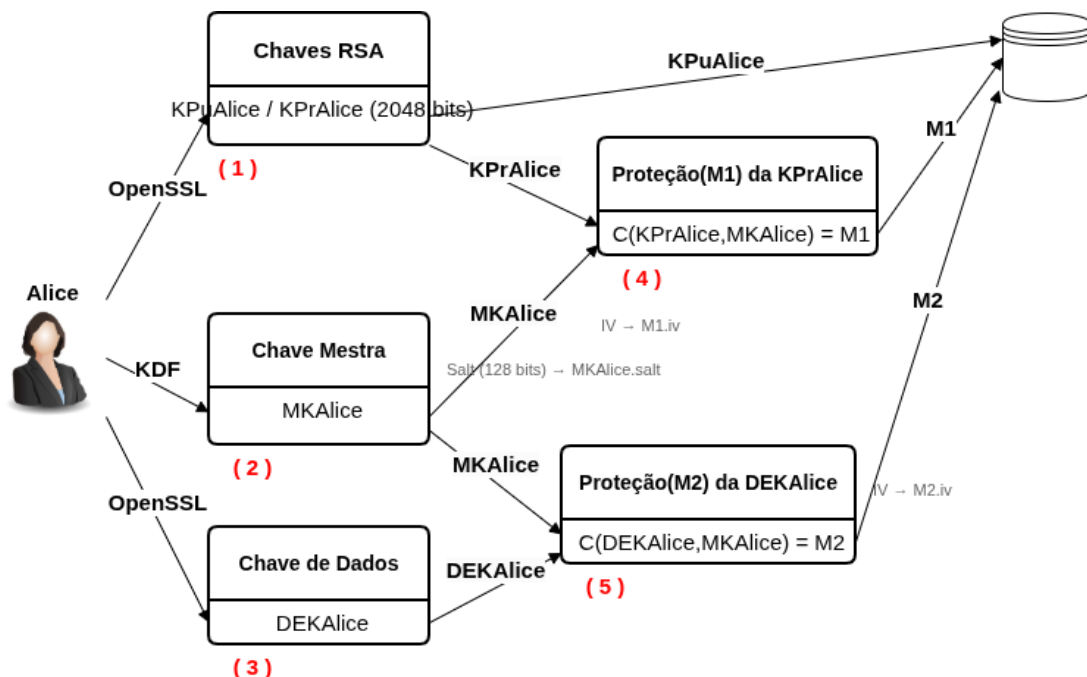


Figura 1: Chaves e Processos associadas a Alice (M1 e M2: ver nota sobre autenticação)

1. Criação da Chave Pública (KPUAlice.pem) e da Chave Privada (KPrAlice.pem);

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out KPrAlice.pem
$ openssl rsa -pubout -in KPrAlice.pem -out KPUAlice.pem
$ openssl req -new -x509 -key KPrAlice.pem -out Alice.cert.pem \
-subj "/CN=Alice"
```

2. Derivação local da Chave Mestra da Alice (MKAlice.hex), baseada na Senha Mestra (MPAlice). O salt de 128 bits é armazenado separadamente





para permitir a derivação durante a recuperação;

```
$ read -rs MASTER_PASSWORD
$ SALT_HEX=$(openssl rand -hex 16)
$ printf '%s' "$SALT_HEX" > MKAlice.salt
$ openssl kdf -keylen 32 -kdfopt digest:SHA256 \
-kdfopt pass:"$MASTER_PASSWORD" -kdfopt salt:"$SALT_HEX" \
-kdfopt iter:600000 PBKDF2 | tr -d ':\n' > MKAlice.hex
```

3. Criação da Chave de Dados (DEKAlice.bin);

```
$ openssl rand -out DEKAlice.bin 32
```

4. A Chave Privada (KPrAlice.pem) é protegida com a Chave Mestra (MKAllice.hex), gerando o Texto Cifrado M1. O IV é armazenado junto com o artefato cifrado;

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M1.iv
$ openssl enc -aes-256-cbc -K "$(cat MKAllice.hex)" -iv "$IV_HEX" \
-in KPrAlice.pem -out M1.bin
```

5. A Chave de Dados (DEKAlice.bin) é protegida com a Chave Mestra (MKAllice.hex), gerando o Texto Cifrado M2 que é enviado para o servidor.

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M2.iv
$ openssl enc -aes-256-cbc -K "$(cat MKAllice.hex)" -iv "$IV_HEX" \
-in DEKAlice.bin -out M2.bin
```

1.2 Bob

Processo de criação das chaves, Pública (KPUBob.pem), Privada (KPrBob.pem) e de Dados (DEKBob.bin), juntamente com o processo de encriptação para armazenamento seguro das chaves Privada (KPrBob.pem) e de Dados (DEKBob.bin).



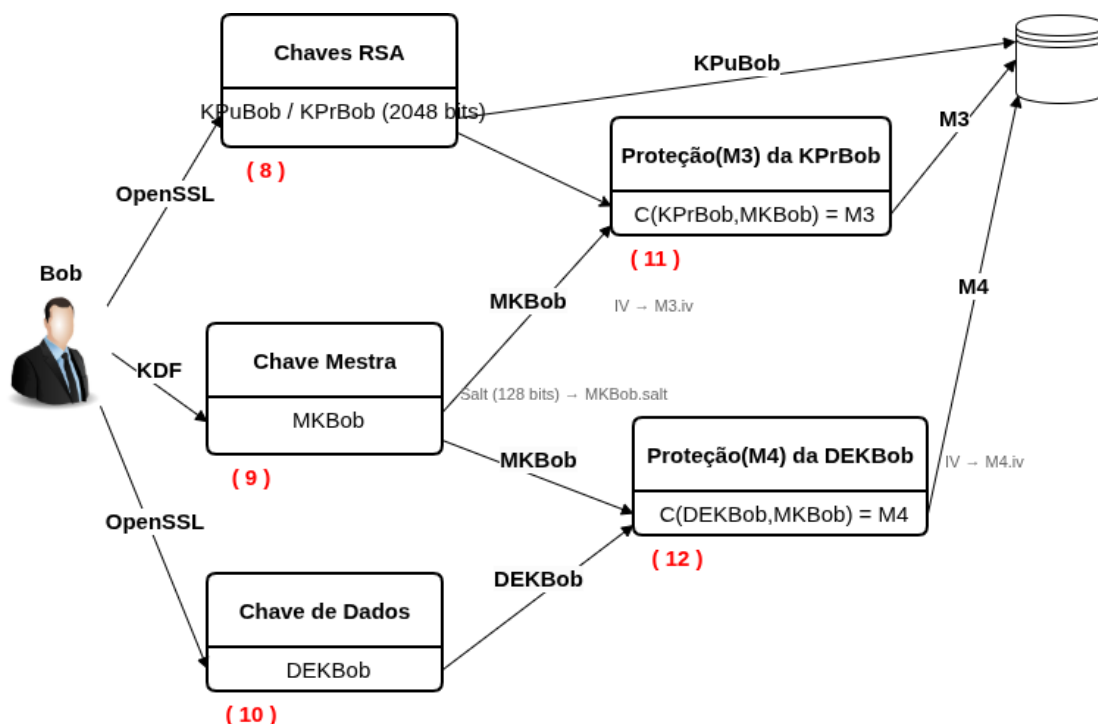


Figura 2: Chaves e Processos associados ao Bob (M3 e M4: ver nota sobre autenticação)

8. Criação das Chave Pública (KPubBob.pem) e da Chave Privada (KPrBob.pem);

```
$ openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out KPrBob.pem
$ openssl rsa -pubout -in KPrBob.pem -out KPubBob.pem
```

9. Derivação local da Chave Mestra (MKBob.hex) baseada na Senha Mestra (MPBob);

```
$ read -rs MASTER_PASSWORD
$ SALT_HEX=$(openssl rand -hex 16)
$ printf '%s' "$SALT_HEX" > MKBob.salt
$ openssl kdf -keylen 32 -kdfopt digest:SHA256 \
-kdfopt pass:"$MASTER_PASSWORD" -kdfopt salt:"$SALT_HEX" \
-kdfopt iter:600000 PBKDF2 | tr -d ':\n' > MKBob.hex
```

10. Criação da Chave de Dados (DEKBob.bin);





```
$ openssl rand -out DEKBob.bin 32
```

11. A Chave Privada (KPrBob) é protegida com a Chave Mestra (MKBob.hex), gerando o Texto Cifrado M3 que é enviado para o servidor;

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M3.iv
$ openssl enc -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$IV_HEX" \
  -in KPrBob.pem -out M3.bin
```

12. A Chave de Dados (DEKBob.bin) é protegida com a Chave Mestra (MKBob.hex), gerando o Texto Cifrado M4 que é enviado para o servidor.

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M4.iv
$ openssl enc -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$IV_HEX" \
  -in DEKBob.bin -out M4.bin
```





2 Custódia das Chaves de Criptografia

No exemplo apresentado, Alice é a usuária Custodiante e Bob é o usuário que terá as suas chaves custodiadas. Assim, todas as chaves do Bob devem ser compartilhadas com a Custodiante Alice.

2.1 Custodiando

Bob irá custodiar as suas duas chaves, Chave Privada(KPrBob.pem) e Chave de Dados(DEKBob.bin), com a custodiante Alice.

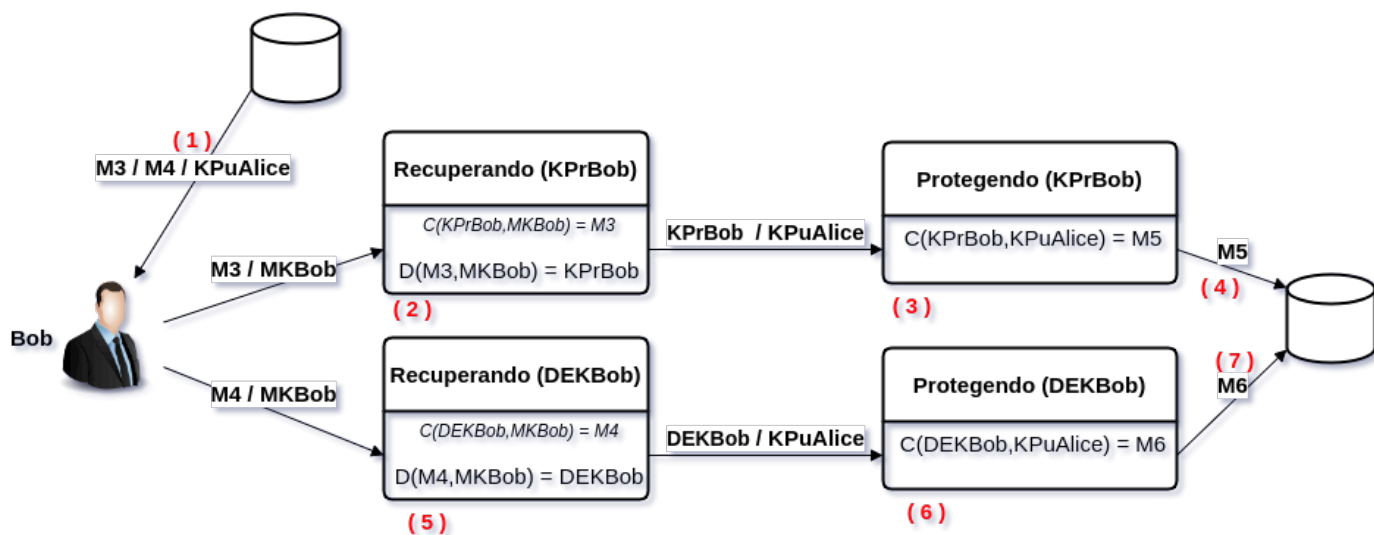


Figura 3: Processos de Custódia das Chaves do Bob

1. Bob recupera os Textos Cifrados M3 e M4 e a Chave Pública da Alice (KPUAlice.pem) do servidor;
2. Bob descriptografa o Texto Cifrado M3 com sua Chave Mestra (MKBob.hex) e o IV associado para obter sua Chave Privada (KPrBob);

```
$ openssl enc -d -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$(cat M3.iv)" \
-in M3.bin -out KPrBob.pem
```





3. Bob protege a sua Chave Privada (KPrBob.pem) para a custodiante Alice, gerando o Texto Cifrado M5. O envelope CMS usa criptografia híbrida: cifra o conteúdo e encapsula a chave de conteúdo para o certificado da Alice (Alice.cert.pem).

```
$ openssl cms -encrypt -binary -in KPrBob.pem -out M5.pem -outform PEM \
-recipient Alice.cert.pem -aes256
```

4. Bob envia o Texto Cifrado M5 para o Servidor;
5. Bob descriptografa o Texto Cifrado M4 com sua Chave Mestra (MKBob.hex) e o IV associado para obter sua Chave de Dados (DEKBob.bin);

```
$ openssl enc -d -aes-256-cbc -K "$(cat MKBob.hex)" -iv "$(cat M4.iv)" \
-in M4.bin -out DEKBob.bin
```

6. Bob criptografa sua Chave de Dados (DEKBob.bin) com a Chave Pública da Alice (KPuAlice.pem), gerando o Texto Cifrado M6.bin;

```
$ openssl pkeyutl -encrypt -inkey KPuAlice.pem -pubin -in DEKBob.bin -out M6.bin \
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

7. Bob envia o Texto Cifrado M6 para o Servidor;





3 Recuperando Custódia das Chaves

Em um cenário em que Bob perde sua Chave Mestra (MKBob.hex), ele solicita à custodiante Alice que criptografe suas duas chaves, Chave Privada (KPr-Bob.pem) e Chave de Dados (DEKBob.bin), com a Nova Chave Mestra (NMK-Bob.hex).

3.1 Gerando Nova Chave Mestra - Bob

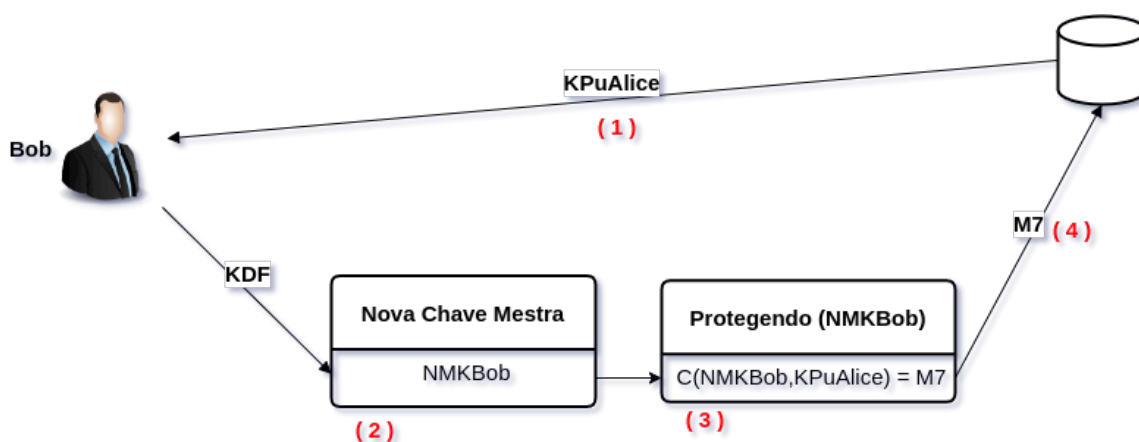


Figura 4: Processos de criação da Nova Chave Mestra do Bob

1. Bob recupera a Chave Pública da Alice (KPUAlice.pem) que está armazenada no servidor;
2. Utilizando uma Key Derivation Function (KDF), Bob gera uma Nova Chave Mestra (NMKBob.hex);

```
$ read -rs MASTER_PASSWORD
$ SALT_HEX=$(openssl rand -hex 16)
$ printf '%s' "$SALT_HEX" > NMKBob.salt
$ openssl kdf -keylen 32 -kdfopt digest:SHA256 \
-kdfopt pass:"$MASTER_PASSWORD" -kdfopt salt:"$SALT_HEX" \
-kdfopt iter:600000 PBKDF2 | tr -d '\n' > NMKBob.hex
```

3. Utilizando a Chave Pública da Alice (KPUAlice.pem), Bob protege sua Nova Chave Mestra (NMKBob.hex), gerando o Texto Cifrado M7;





```
$ openssl pkeyutl -encrypt -inkey KPuAlice.pem -pubin -in NMKBob.hex -out M7.bin \
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

4. O Texto Cifrado M7 é enviado para o servidor.

3.2 Nova Custódia - Alice

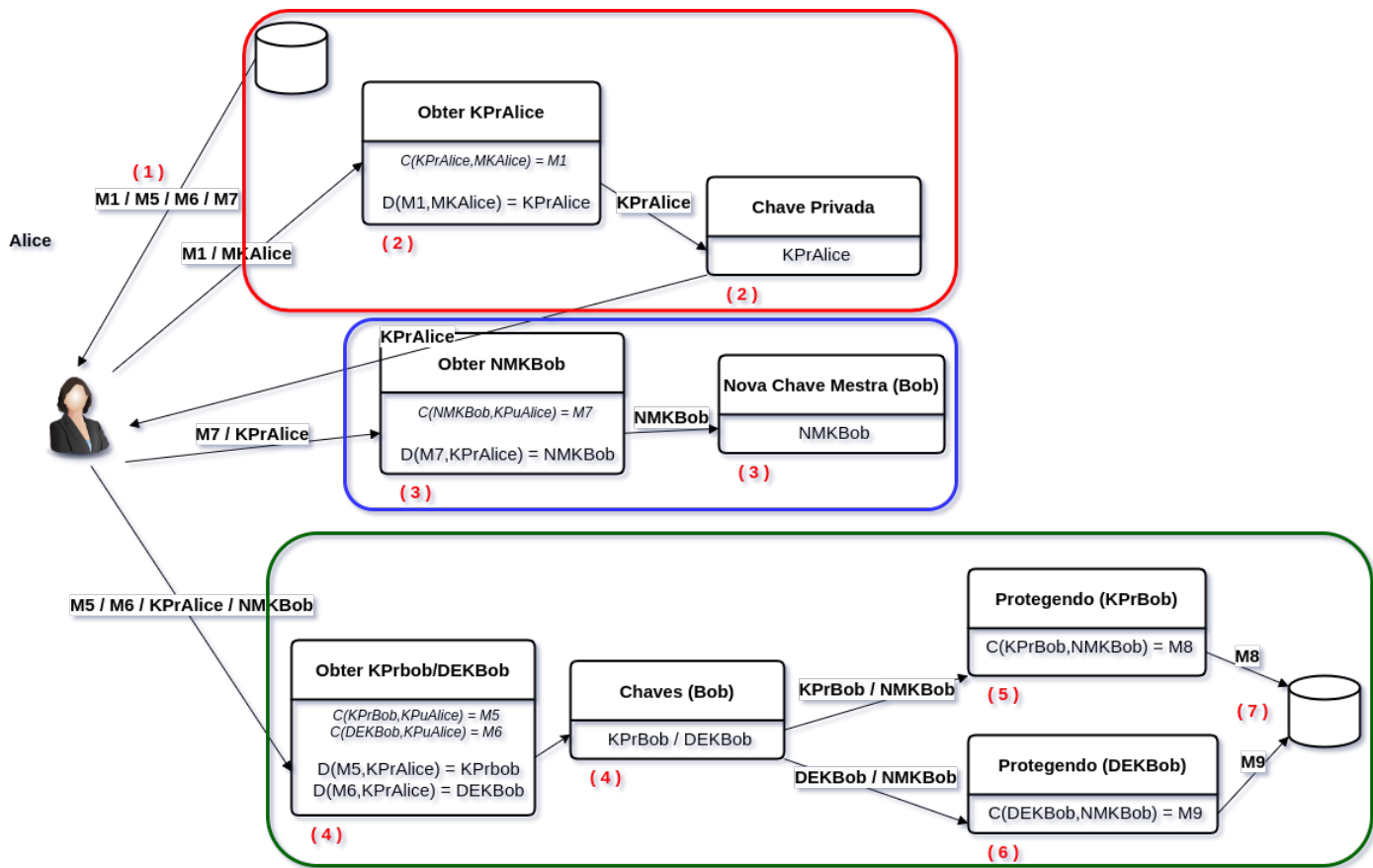


Figura 5: Processos de proteção das chaves do Bob com a sua Nova Chave Mestra

1. Alice faz o download dos Textos Cifrados M1, M5, M6 e M7 do servidor;
2. Alice descriptografa o Texto Cifrado M1 com sua Chave Mestra (MKAlice.hex) e o IV associado para obter sua Chave Privada (KPrAlice.pem);





```
$ openssl enc -d -aes-256-cbc -K "$(cat MKAlice.hex)" -iv "$(cat M1.iv)" \
-in M1.bin -out KPrAlice.pem
```

3. Com sua Chave Privada (KPrAlice.pem), Alice descriptografa o Texto Cifrado M7 e obtém a Nova Chave Mestra (NMKBob.hex) do Bob;

```
$ openssl pkeyutl -decrypt -inkey KPrAlice.pem -in M7.bin -out NMKBob.hex \
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

4. Com sua Chave Privada (KPrAlice.pem), Alice recupera os artefatos de custódia M5 e M6 e obtém a Chave Privada do Bob (KPrBob.pem) e a Chave de Dados do Bob (DEKBob.bin);

```
$ openssl cms -decrypt -in M5.pem -inform PEM -recip Alice.cert.pem \
-inkey KPrAlice.pem -out KPrBob.pem

$ openssl pkeyutl -decrypt -inkey KPrAlice.pem -in M6.bin -out DEKBob.bin \
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256
```

5. Com a Nova Chave Mestra do Bob (NMKBob.hex), Alice criptografa a Chave Privada do Bob (KPrBob.pem) gerando o Texto Cifrado M8;

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M8.iv
$ openssl enc -aes-256-cbc -K "$(cat NMKBob.hex)" -iv "$IV_HEX" \
-in KPrBob.pem -out M8.bin
```

6. Com a Nova Chave Mestra do Bob (NMKBob.hex), Alice criptografa a Chave de Dados do Bob (DEKBob.bin) gerando o Texto Cifrado M9;

```
$ IV_HEX=$(openssl rand -hex 16)
$ printf '%s' "$IV_HEX" > M9.iv
$ openssl enc -aes-256-cbc -K "$(cat NMKBob.hex)" -iv "$IV_HEX" \
-in DEKBob.bin -out M9.bin
```

7. Os Textos Cifrados M8 e M9 são enviados para o servidor.

A partir deste ponto, a Chave Privada do Bob (KPrBob.pem) e a Chave de Dados do Bob (DEKBob.bin) estão protegidas com a Nova Chave Mestra do





Bob (NMKBob.hex) e estão armazenadas no servidor. Os IVs M8.iv e M9.iv acompanham os respectivos textos cifrados.





Copyright

All rights reserved to MT4 Tecnologia LTDA.

Propósito deste documento

New MySafe - Account Recovery

Versão

Versão 1.0

Autor

Caio Abreu Ferreira [cferreiras@senhasegura.com]

MT4 Tecnologia Ltda.

Rua Joaquim Antunes, 767 – Pinheiros – São Paulo

Phone: +55 11 3065-7070

www.mt4.com.br

